

RED HAT CERTIFIED SPECIALIST IN CONTAINERS

EX188

Guia de Estudos Completo

6

Simulados

90

Questoes

35

Topicos

Systemd & Quadlet

Storage & Volumes

Networking Avancado

Build & Secrets

Registry & Skopeo

Troubleshooting

Índice de Conteúdo

■ S1 — Fundamentos & Bases

- Q1 Run a Container as a Systemd Service
- Q2 Configure a Restart Policy
- Q3 Manage Rootless Host Volumes
- Q4 Build Image with Custom User
- Q5 Use Podman Secrets
- Q6 Build Arguments ARG vs ENV
- Q7 Create Application Stack - Podman Compose
- Q8 Create a Pod
- Q9 Push to Insecure Local Registry
- Q10 Analyze Container Events
- Q11 Inspect Internal Application State
- Q12 Limit Container Resources
- Q13 Clean Up Pruning
- Q14 Generate Systemd for Existing Pod
- Q15 Save and Load an Image

■ S2 — Build Avançado & Storage

- Q1 Build Configurable Image
- Q2 Implement Image Labels
- Q3 Copy and Extract Archives (ADD vs COPY)
- Q4 Persistent Database Storage
- Q5 Rootless Host Bind Mounts
- Q6 Deploy Multi-Container App Compose
- Q7 Container Name Resolution DNS
- Q8 Auto-Start with Systemd Quadlet
- Q9 Enforce Memory Limits
- Q10 Inject Secrets
- Q11 Run as Non-Standard User
- Q12 Resolve Port Conflicts
- Q13 Inspect Container Logs
- Q14 Prune Unused Objects
- Q15 Tag and Push to Local Registry

■ S3 — Pods & Kubernetes

- Q1 Create Pod with Port Mapping
 - Q2 Deploy Containers into a Pod
 - Q3 Generate Systemd Service for Pod
 - Q4 Create Custom Network with Subnet
 - Q5 Assign Static IP
 - Q6 Troubleshoot DNS Resolution
 - Q7 Sidecar Pattern - Shared Pod Volume
-

- Q8 Container Health Checks
- Q9 Generate Kubernetes YAML
- Q10 Import Kubernetes YAML
- Q11 Remote Image Inspection with Skopeo
- Q12 Copy Images Between Transports
- Q13 Multi-Stage Build Optimization
- Q14 ENTRYPOINT vs CMD
- Q15 Capture Container Death Events

■ S4 — Networking Avançado & Segurança

- Q1 Custom Gateway
- Q2 Multi-Homed Container
- Q3 Troubleshoot Protocol Mismatch UDP
- Q4 Deploy from Kubernetes YAML Manifest
- Q5 Auto-Start Kubernetes Pod with Quadlet (.kube)
- Q6 Inject Secrets at Custom Path
- Q7 Harden Image - Remove ALL SUID Binaries
- Q8 Share Process Namespace
- Q9 Limit Container Disk I/O
- Q10 Build from Tarball Context
- Q11 Remote Image Inspection Skopeo
- Q12 Optimize Image Layers
- Q13 ENTRYPOINT vs CMD (Pinger)
- Q14 Container as Build Environment
- Q15 Fix Container Health Check

■ S5 — Troubleshooting Avançado

- Q1 Missing Host Directory
- Q2 Fix Broken Containerfile
- Q3 Troubleshoot Pod Communication
- Q4 Troubleshoot SELinux Permission Denied
- Q5 Harden Image - Remove SUID
- Q6 Fix Name Resolution Failure
- Q7 Troubleshoot Quadlet Service
- Q8 Fix Build Dependency Order
- Q9 Troubleshoot play kube Failure
- Q10 Fix Failing Health Check
- Q11 Fix Secret Mounting (Custom Path)
- Q12 Troubleshoot Bind Mount Shadowing
- Q13 Troubleshoot Registry Push TLS
- Q14 Monitor Container Events Forensics
- Q15 Rootless Storage UID Mapping

■ S6 — Troubleshooting & Avançado

- Q1 Pod Port Mismatch
 - Q2 Broken Network Isolation
 - Q3 SELinux Port Denial
-

- Q4 Optimize Inefficient Containerfile
- Q5 Fix Broken Entrypoint Script
- Q6 Manual Build with Buildah
- Q7 FROM scratch Build
- Q8 Rootless Port Binding (porta 80)
- Q9 Fix Rootless Host Permission
- Q10 Debug Kubernetes YAML
- Q11 Remote Inspection Skopeo
- Q12 Troubleshoot Flapping Health Check
- Q13 Complex Data Pipeline
- Q14 Audit Container Exec
- Q15 Capstone: Gitea + PostgreSQL

Checklist Final — 35 Tópicos

S1

Fundamentos & Bases

Simulado 1 · 15 Questões

S1Q1

Run a Container as a Systemd Service

web_service, httpd, 8080:80, auto-start

■ Objetivo

Fazer o container web_service (httpd) iniciar automaticamente no boot, mesmo sem ninguém logado.

■ Passo a passo

Método MODERNO — Quadlet (RHEL 9+):

```
loginctl enable-linger student
mkdir -p ~/.config/containers/systemd

# Criar: ~/.config/containers/systemd/web_service.container
[Container]
Image=docker.io/library/httpd
ContainerName=web_service
PublishPort=8080:80

[Install]
WantedBy=default.target

systemctl --user daemon-reload
systemctl --user enable --now web_service
```

BODY

loginctl enable-linger permite que serviços do usuário rodem sem login ativo. O arquivo .container é lido pelo Podman como uma unit systemd automaticamente. A seção [Install] WantedBy=default.target é obrigatória para o boot.

■ ■ Esquecer o [Install] faz o serviço funcionar manualmente mas NÃO iniciar no boot. O --new é crítico no método legado generate systemd.

S1Q2 Configure a Restart Policy`resilient_app, --restart=on-failure`**■ Objetivo**

Container que reinicia sozinho se crashar (exit code diferente de zero).

```
podman run -d --name resilient_app --restart=on-failure \
    registry.access.redhat.com/ubi9/ubi sleep 3600

podman exec resilient_app kill 1 # simular crash
podman ps                       # verificar que reiniciou
```

■ Diferenças

--restart=on-failure reinicia apenas se exit ≠ 0. --restart=always reinicia sempre, inclusive após podman stop. --restart=unless-stopped igual ao always exceto se parado manualmente.

■ **Restart policy sozinha NÃO garante boot persistence. Para isso, use Quadlet/systemd.**

S1Q3 Manage Rootless Host Volumes`db_writer, UID 2000, ~/database_data:/data`**■ Objetivo**

Container rodando como UID 2000 precisa escrever em um diretório do host em modo rootless.

```
mkdir ~/database_data
podman unshare chown 2000:2000 ~/database_data

podman run -d --name db_writer --user 2000 \
    -v ~/database_data:/data:Z \
    registry.access.redhat.com/ubi9/ubi \
    sh -c 'touch /data/success.txt && sleep 3600'
```

■ Por que podman unshare

Em modo rootless, UID 2000 dentro do container mapeia para um UID diferente no host. O podman unshare executa dentro do mesmo namespace que o Podman usa, então chown 2000:2000 ajusta corretamente.

■ **:Z (maiúsculo) é privado — só este container. :z (minúsculo) é compartilhado. NUNCA use setenforce 0.**

S1Q4

Build Image with Custom User

appgroup GID 5000, appuser UID 4000

■ Objetivo

Imagem com grupo e usuário específicos por GID/UID numérico para compatibilidade com OpenShift.

```
FROM registry.access.redhat.com/ubi9/ubi
RUN groupadd -g 5000 appgroup && \
    useradd -u 4000 -g appgroup appuser
USER 4000:5000
```

```
podman build -t secure-app:1.0 .
```

■ Por que IDs numéricos

OpenShift e Kubernetes exigem IDs numéricos na instrução USER. USER appuser pode não resolver em contextos de segurança do cluster. USER 4000:5000 é a forma correta e portátil.

■ **Sempre crie o grupo antes do usuário — useradd referencia o grupo que já deve existir.**

S1Q5

Use Podman Secrets

db_password=SuperSecret123

■ Objetivo

Injetar senhas no container sem expô-las em variáveis de ambiente ou na imagem.

```
printf 'SuperSecret123' | podman secret create db_password -

podman run -d --name secured_db --secret db_password \
    registry.access.redhat.com/ubi9/ubi sleep 3600

podman exec secured_db cat /run/secrets/db_password
```

■ Como funciona

O secret é montado como arquivo em /run/secrets/ — um tmpfs em memória, não vai para o disco. Muito mais seguro que variáveis de ambiente que aparecem em podman inspect.

■ **Use printf (não echo) — echo adiciona \n invisível que quebra autenticação de aplicações que leem senha byte a byte.**

S1Q6

Build Arguments ARG vs ENV

versioned-app, APP_VERSION=2.5

■ Objetivo

Entender a diferença entre ARG (build-time only) e ENV (persiste em runtime).

```
FROM registry.access.redhat.com/ubi9/ubi
ARG APP_VERSION
RUN echo $APP_VERSION > /app_version.txt

podman build --build-arg APP_VERSION=2.5 -t versioned-app .
```

■ Diferença fundamental

ARG existe apenas durante o build — não aparece no container em execução. ENV persiste no container e fica visível em `podman inspect` e via `env` no terminal. Use ARG para versões de pacotes, URLs. Use ENV para config de runtime.

■ **ARG com valor padrão: ARG APP_VERSION=1.0 — se não passar --build-arg, usa o padrão.**

S1Q7

Create Application Stack - Podman Compose

nginx + ubi + volume + rede

■ Objetivo

Orquestrar múltiplos containers com volumes e rede compartilhada usando Compose.

```
# docker-compose.yml
version: '3'
services:
  frontend:
    image: docker.io/library/nginx:latest
    volumes: [data_vol:/var/www/html]
    networks: [micronet]
  backend:
    image: registry.access.redhat.com/ubi9/ubi
    command: sleep 3600
    networks: [micronet]
volumes:
  data_vol:
networks:
  micronet:

podman compose up -d # RHEL9+: espaço, não hífen
```

■ Pontos críticos

No RHEL 9+, o comando é podman compose (com espaço). Os dois serviços ficam na rede micronet e se comunicam pelo nome do serviço. O volume data_vol é nomeado e persistente entre restarts.

■ ■ podman-compose (com hífen) é um pacote separado e pode não estar instalado. Use podman compose nativo.

S1Q8**Create a Pod**

web-pod, 8080:80, nginx

■ Objetivo

Criar um Pod e executar container dentro dele. Portas são definidas no Pod, não nos containers.

```
podman pod create --name web-pod -p 8080:80
podman run -d --name web-container --pod web-pod docker.io/library/nginx
curl localhost:8080
```

■ Conceito fundamental

O Pod cria um container de infraestrutura (infra) que segura o namespace de rede. Containers dentro compartilham rede e se enxergam via localhost. Mapeamento de portas é definido APENAS no Pod.

■ Usar **-p** no podman run dentro de um pod resulta em erro. Nunca especifique portas nos containers filhos.

S1Q9**Push to Insecure Local Registry**

localhost:5000, --tls-verify=false

■ Objetivo

Enviar imagem para um registry local sem TLS (desenvolvimento/lab).

```
podman tag registry.access.redhat.com/ubi9/ubi localhost:5000/my-ubi
podman push --tls-verify=false localhost:5000/my-ubi

# Configuração permanente em /etc/containers/registries.conf:
[[registry]]
location = 'localhost:5000'
insecure = true
```

■ Passo a passo

1. tag cria alias com endereço do registry no nome. 2. push --tls-verify=false ignora verificação. 3. Configuração permanente evita o flag toda vez.

■ Para configuração segura com certificado, use /etc/containers/certs.d/ca.crt — exigido quando o exame pede modo seguro.

S1Q10 Analyze Container Events

stream + grep died

■ Objetivo

Monitorar eventos em tempo real e filtrar containers que morreram.

```
podman events --stream > events.log &
podman run --rm --name test-event \
  registry.access.redhat.com/ubi9/ubi echo 'hello'
grep 'died' events.log
fg # Ctrl+C para encerrar
```

■ Tipos de eventos

died = crash ou kill (exit ≠ 0). exited = saída limpa (exit = 0). exec = alguém executou podman exec (útil para auditoria). stopped = parado administrativamente.

■ *O & coloca o monitor em background. fg o traz de volta antes de Ctrl+C.*

S1Q11 Inspect Internal Application State

exec + curl

■ Objetivo

Verificar se aplicação dentro do container está funcionando sem expor portas.

```
podman run -d --name web_test registry.access.redhat.com/ubi9/httpd-24
podman exec web_test curl -s localhost
```

■ Por que isso importa

podman exec executa dentro do namespace do container — mesma rede, mesmo filesystem. Útil para testar conectividade interna, verificar configs geradas em runtime, ou debugar sem expor portas.

■ *Se o container não tiver curl, use: cat /proc/net/tcp para ver portas abertas, ou wget como alternativa.*

S1Q12 Limit Container Resources

limited-app, 256MB

■ Objetivo

Impedir que container consuma toda a memória do host.

```
podman run -d --name limited-app --memory 256m \
registry.access.redhat.com/ubi9/ubi sleep 3600

podman inspect --format '{{.HostConfig.Memory}}' limited-app
# Resultado: 268435456 (256 x 1024 x 1024 bytes)
```

■ Outros limites

--cpus 0.5 limita a meio núcleo de CPU. --memory-swap igual a --memory desabilita swap. --pids-limit protege contra fork bomb.

■ Se o container ultrapassar o limite, o kernel o mata com OOMKilled (Out of Memory Killed).

S1Q13 Clean Up Pruning

por tipo, com -f

■ Objetivo

Remover objetos não utilizados de forma granular para liberar espaço em disco.

```
podman stop -a                # para todos os containers
podman container prune -f      # remove containers parados
podman image prune -a -f       # remove imagens sem container
podman system prune -f         # tudo (menos volumes)
podman system prune -f --volumes # tudo incluindo volumes
```

■ Ordem recomendada

Parar containers primeiro (em execução não podem ser removidos). Depois containers, depois imagens. system prune é atalho para tudo de uma vez.

■ --volumes apaga dados de banco de dados! Use com consciência. image prune sem -a remove só dangling (sem tag).

S1Q14 Generate Systemd for Existing Pod

web-pod → pod-web-pod.service

■ Objetivo

Criar unit files systemd para um pod existente e habilitá-lo no boot.

```
mkdir -p ~/.config/systemd/user && cd ~/.config/systemd/user
podman generate systemd --new --name web-pod --files
systemctl --user daemon-reload
systemctl --user enable --now pod-web-pod.service
loginctl enable-linger student
```

■ O que --new faz

Sem --new: inicia container já existente — se for apagado, serviço quebra. Com --new: cria novo container toda vez que o serviço inicia — muito mais robusto para boot persistence.

■ *Para pods, o prefixo gerado é pod-. Então web-pod vira pod-web-pod.service.*

S1Q15 Save and Load an Image

ubi_backup.tar

■ Objetivo

Exportar imagem para arquivo tar e reimportá-la — essencial para ambientes air-gapped.

```
podman save -o /tmp/ubi_backup.tar registry.access.redhat.com/ubi9/ubi
podman rmi registry.access.redhat.com/ubi9/ubi
podman load -i /tmp/ubi_backup.tar
```

■ save/load vs export/import

save/load preserva TODAS as layers e metadados (histórico, variáveis, entrypoint). export/import exporta só o filesystem do container, perde metadados.

■ *No exame, use sempre save/load para imagens completas. O ambiente de prova pode ser air-gapped (sem internet).*

S2

Build Avançado & Storage

Simulado 2 · 15 Questões

S2Q1

Build Configurable Image

ARG APP_VER=1.0, ENV release=stable, configurable-app:2.0

■ Objetivo

Criar imagem com ARG (valor padrão sobrescritível) + ENV (persistente em runtime).

```
FROM registry.access.redhat.com/ubi9/ubi-minimal
ARG APP_VER=1.0
ENV release=stable
RUN echo "$APP_VER" > /opt/version

podman build --build-arg APP_VER=2.0 -t configurable-app:2.0 .
```

■ Combinação ARG+ENV

ARG APP_VER=1.0 define valor padrão. --build-arg APP_VER=2.0 sobrescreve em tempo de build. ENV release=stable fica disponível no container (echo \$release retorna stable).

■■ ARG definido ANTES do FROM só existe para o FROM. ARG depois do FROM existe para os RUN seguintes na mesma stage.

S2Q2**Implement Image Labels**

maintainer=student, usage=web-server

■ Objetivo

Adicionar metadados à imagem para identificação, filtragem e documentação.

```
FROM registry.access.redhat.com/ubi9/ubi-minimal
LABEL maintainer="student" usage="web-server"
RUN echo "1.0" > /opt/version

podman build -t labeled-app:latest .
podman inspect --format '{{.Labels.usage}}' labeled-app:latest
# Output: web-server
```

■ Por que Labels importam

Filtrar imagens: `podman images --filter label=usage=web-server`. Em OpenShift, labels selecionam recursos. São metadados que não afetam comportamento.

■ Labels são uma boa prática — identifique versão, mantenedor e propósito em toda imagem de produção.

S2Q3**Copy and Extract Archives (ADD vs COPY)**

ADD extrai .tar.gz automaticamente, COPY apenas copia

■ Objetivo

Saber quando usar ADD (extração automática) no lugar de COPY.

```
echo 'Hello' > index.html
tar -czf app_src.tar.gz index.html

# Containerfile — ADD extrai automaticamente
FROM registry.access.redhat.com/ubi9/ubi
ADD app_src.tar.gz /var/www/html/

podman build -t web-content:1.0 .
```

■ Regra de uso

COPY: copia arquivos, simples e previsível. ADD: extrai .tar.gz automaticamente e pode baixar URLs. Se a tarefa diz 'extract' ou o arquivo é .tar.gz, use ADD. Para todo o resto, prefira COPY.

■ ADD com URL é considerado má prática — sem controle de checksums. Prefira RUN curl para downloads.

S2Q4**Persistent Database Storage**`mysql_data, mariadb, /var/lib/mysql`**■ Objetivo**

Volume nomeado para dados de banco que sobrevivem ao container ser removido.

```
podman volume create mysql_data
podman run -d --name mydb \
-v mysql_data:/var/lib/mysql \
-e MARIADB_ROOT_PASSWORD=redhat \
docker.io/library/mariadb
```

■ Named Volume vs Bind Mount

Named Volume: gerenciado pelo Podman, SELinux automático, primeira execução copia dados da imagem. Bind Mount: você gerencia, precisa de :Z para SELinux, sobrescreve conteúdo completamente.

■ Named Volumes NÃO precisam de :Z — Podman aplica SELinux label automaticamente. Bind mounts sempre precisam.

S2Q5**Rootless Host Bind Mounts**`~/audit_logs:/var/log/audit:Z`**■ Objetivo**

Montar diretório do host em container rootless com SELinux corretamente configurado.

```
mkdir ~/audit_logs
podman run --rm --name logger \
-v ~/audit_logs:/var/log/audit:Z \
registry.access.redhat.com/ubi9/ubi \
sh -c 'date > /var/log/audit/testaudit'
cat ~/audit_logs/testaudit
```

■ :Z vs :z

Z (maiúsculo) = label privado, só este container pode acessar. z (minúsculo) = label compartilhado, múltiplos containers podem acessar o mesmo diretório.

■ NUNCA use setenforce 0 no exame — você perde pontos automaticamente e não resolve o problema real.

S2Q6

Deploy Multi-Container App Compose

web:8080 + db:mariadb + app_net

■ Objetivo

Stack completo com web, banco de dados, rede isolada e volume persistente.

```
version: '3'
services:
  web:
    image: docker.io/library/nginx
    ports: ["8080:80"]
    networks: [app_net]
  db:
    image: docker.io/library/mariadb
    environment:
      MARIADB_ROOT_PASSWORD: secret
    networks: [app_net]
    volumes: [db_data:/var/lib/mysql]
networks:
  app_net:
volumes:
  db_data:
```

podman compose up -d

■ Comunicação entre serviços

web e db estão em app_net — web alcança db pelo hostname db (nome do serviço). db_data é named volume que persiste dados do MariaDB entre restarts.

■ *Sempre defina rede explícita — isso isola os serviços de outros containers no host.*

S2Q7

Container Name Resolution DNS

DNS só funciona em rede custom

■ Objetivo

Demonstrar que DNS por nome só funciona em redes custom, não na rede padrão.

```
podman network create internal-net
podman run -d --name server --network internal-net \
    registry.access.redhat.com/ubi9/ubi sleep 600

# Air-gapped safe (sem instalar pacotes):
podman run --rm --network internal-net \
    registry.access.redhat.com/ubi9/ubi getent ahosts server
```

■ Por que getent ahosts

getent ahosts usa o resolver do sistema — já disponível na UBI sem instalar nada. Em ambientes air-gapped (sem internet), instalar pacotes pode não ser possível.

■ Na rede padrão do Podman, containers se enxergam apenas por IP, não por nome. Em redes custom, DNS interno resolve nomes automaticamente.

S2Q8**Auto-Start with Systemd Quadlet**

auto-web, nginx, porta 80 rootless

■ Objetivo

Nginx na porta 80 como usuário rootless com auto-start no boot.

```
echo 'net.ipv4.ip_unprivileged_port_start=80' | \
  sudo tee /etc/sysctl.d/99-rootless.conf
sudo systemctl --system
logindctl enable-linger student

# auto-web.container:
[Container]
ContainerName=auto-web
Image=docker.io/library/nginx
PublishPort=80:80

[Install]
WantedBy=default.target

systemctl --user daemon-reload
systemctl --user enable --now auto-web.service
```

■ Por que precisa do sysctl

Portas abaixo de 1024 são privilegiadas. Usuário rootless não pode usá-las por padrão.

■ **sysctl -w é temporário (perde no reboot). SEMPRE escreva em /etc/sysctl.d/*.conf para configuração permanente.**

S2Q9

Enforce Memory Limits

limited-db, 512MB, mariadb

■ Objetivo

Limitar o uso de memória do MariaDB para proteger o host.

```
podman run -d --name limited-db --memory 512m \
-e MARIADB_ROOT_PASSWORD=pass \
docker.io/library/mariadb

podman inspect --format '{{.HostConfig.Memory}}' limited-db
# 536870912 = 512 x 1024 x 1024 bytes
```

■ Verificação

podman stats limited-db mostra uso em tempo real. Se ultrapassar: OOMKilled. podman inspect retorna valor em bytes.

■ **Sufixos aceitos: 256m, 1g, 512k. Para desabilitar swap: --memory-swap igual a --memory.**

S2Q10

Inject Secrets

db_creds=SuperSafePassword, secure-app

■ Objetivo

Criar e montar secret de forma segura, sem expor em variáveis de ambiente.

```
printf 'SuperSafePassword' | podman secret create db_creds -
podman run -d --name secure-app --secret db_creds \
registry.access.redhat.com/ubi9/ubi sleep 3600
podman exec secure-app cat /run/secrets/db_creds
```

■ **Sempre use printf em vez de echo. O \n invisível do echo corrompe senhas em aplicações que leem byte a byte. Secret fica em /run/secrets/ — tmpfs em memória.**

S2Q11 Run as Non-Standard User

security-check, UID 1234

■ Objetivo

Executar container com UID específico para reduzir risco de segurança.

```
podman run -d --name security-check --user 1234 \
  registry.access.redhat.com/ubi9/ubi sleep 3600
podman exec security-check id -u
# Output: 1234
```

■ Por que importa

Containers rodando como root (UID 0) representam risco — fuga de container dá acesso root ao host. Em OpenShift, containers que exigem root são rejeitados pelas Security Context Constraints (SCC).

S2Q12 Resolve Port Conflicts

web-alpha:9090, web-beta:9091

■ Objetivo

Diagnosticar e resolver conflito de porta entre dois containers.

```
podman run -d --name web-alpha -p 9090:80 docker.io/library/nginx
# web-beta na mesma porta falha: address already in use
podman run -d --name web-beta -p 9091:80 docker.io/library/nginx

podman ps          # coluna PORTS
podman port web-alpha
```

■ Regra

A porta do HOST deve ser única. A porta do CONTAINER pode repetir (ambos usam 80 internamente). Mensagem de erro: Error: address already in use.

S2Q13 Inspect Container Logs

container crashado, crasher

■ Objetivo

Recuperar logs de container que já parou por erro.

```
podman run -d --name crasher \  
  registry.access.redhat.com/ubi9/ubi \  
  sh -c "echo 'Critical Error'; exit 1"  
podman logs crasher  
# Output: Critical Error
```

■ Opções de logs

podman logs -f: follow em tempo real. podman logs --tail 50: últimas 50 linhas. podman logs --since 10m: últimos 10 minutos. Logs disponíveis mesmo após container parar.

S2Q14 Prune Unused Objects

container/network/image/volume

■ Objetivo

Limpeza granular por tipo de objeto para liberar espaço.

```
podman container prune -f    # containers parados  
podman network prune -f     # redes sem container  
podman image prune -f       # imagens dangling  
podman volume prune -f      # volumes sem container
```

■ Diferenças

image prune sem -a: só dangling (sem tag). image prune -a: todas sem container rodando. system prune: atalho container + network + image dangling. system prune --volumes: inclui volumes (cuidado com dados!).

S2Q15 Tag and Push to Local Registry

web-content:1.0 → localhost:5000

■ Objetivo

Subir registry local e publicar imagem nele.

```
podman run -d -p 5000:5000 --name registry docker.io/library/registry:2
podman tag web-content:1.0 localhost:5000/web-content:1.0
podman push --tls-verify=false localhost:5000/web-content:1.0

# Verificar:
curl http://localhost:5000/v2/web-content/tags/list
```

■ Passo a passo

1. Sobe registry na porta 5000. 2. tag cria alias com endereço do registry no nome. 3. push envia. Para verificar: curl na API v2 do registry.

S3

Pods & Kubernetes

Simulado 3 · 15 Questões

S3Q1

Create Pod with Port Mapping

webapp_pod, 8080:80, 8443:443

■ Objetivo

Pod com múltiplas portas mapeadas — template para aplicações web.

```
podman pod create --name webapp_pod -p 8080:80 -p 8443:443
podman pod ps
```

■ Conceito

O pod cria container infra que segura o namespace de rede. Múltiplos -p adicionam múltiplas portas. Todos os containers herdam esse namespace. Status inicial: Created com 1 container (infra oculto).

S3Q2

Deploy Containers into a Pod

frontend nginx:alpine + backend ubi-minimal

■ Objetivo

Adicionar frontend e backend ao pod, sem especificar portas nos containers.

```
podman run -d --name frontend --pod webapp_pod docker.io/library/nginx:alpine
podman run -d --name backend --pod webapp_pod \
  registry.access.redhat.com/ubi9/ubi-minimal sleep 3600
podman ps --filter pod=webapp_pod
```

■ ■ **NUNCA use -p ao adicionar containers a um pod — as portas já foram definidas no pod. Os containers se comunicam entre si via localhost (compartilham namespace de rede).**

S3Q3

Generate Systemd Service for Pod

webapp_pod, restart-policy=on-failure

■ Objetivo

Gerar unit files systemd para o pod completo com restart policy.

```
mkdir -p ~/.config/systemd/user && cd ~/.config/systemd/user
podman generate systemd --new --files --name webapp_pod \
  --restart-policy=on-failure
systemctl --user daemon-reload
systemctl --user enable --now pod-webapp_pod.service
loginctl enable-linger student
```

■ O que é gerado

Múltiplos arquivos — um para o pod e um para cada container. O serviço do pod gerencia os containers na ordem correta automaticamente.

S3Q4

Create Custom Network with Subnet

secure_net, 192.168.50.0/24

■ Objetivo

Rede com subnet específica para controle de endereçamento IP.

```
podman network create --subnet 192.168.50.0/24 secure_net
podman network inspect secure_net
podman network inspect \
  --format '{{(index .Subnets 0).Gateway}}' secure_net
```

■ Por que definir subnet

Permite atribuir IPs estáticos, configurar firewall por range e isolar tráfego. O gateway padrão é o .1 da subnet (192.168.50.1).

S3Q5

Assign Static IP

`static_host, 192.168.50.10, --ip`

■ Objetivo

Container com IP fixo para serviços que precisam de endereço previsível.

```
podman run -d --name static_host --net secure_net --ip 192.168.50.10 \
registry.access.redhat.com/ubi9/ubi-minimal sleep 3600

podman inspect --format \
'{{.NetworkSettings.Networks.secure_net.IPAddress}}' static_host
```

■ ■ **--ip só funciona em redes CUSTOM. Na rede padrão do Podman, retorna erro. O IP deve estar dentro da subnet da rede criada.**

S3Q6

Troubleshoot DNS Resolution

`getent ahosts, air-gapped safe`

■ Objetivo

Verificar resolução DNS entre containers sem instalar pacotes extras.

```
podman run -d --name client_host --net secure_net \
registry.access.redhat.com/ubi9/ubi-minimal sleep 3600

podman exec -it client_host getent ahosts static_host
```

■ ***getent ahosts é a ferramenta certa para air-gapped — não requer instalação, usa o resolver do SO e confirma que o DNS interno do Podman está funcionando.***

S3Q7

Sidecar Pattern - Shared Pod Volume

logger_pod, writer+reader, shared_data

■ Objetivo

Dois containers no mesmo pod compartilhando dados via volume explícito.

```
podman pod create --name logger_pod
podman volume create shared_data

podman run -d --pod logger_pod --name writer -v shared_data:/mnt/data \
registry.access.redhat.com/ubi9/ubi-minimal \
sh -c 'while true; do date >> /mnt/data/log.txt; sleep 5; done'

podman run -d --pod logger_pod --name reader -v shared_data:/mnt/data \
registry.access.redhat.com/ubi9/ubi-minimal \
sh -c 'tail -f /mnt/data/log.txt'
podman logs reader
```

■ Containers no mesmo pod **NÃO** compartilham filesystem automaticamente — apenas rede. Para compartilhar arquivos, volume explícito montado nos dois containers é **OBRIGATÓRIO**.

S3Q8

Container Health Checks

health_test, nginx, curl a cada 10s

■ Objetivo

Container com probe automático de saúde monitorando a aplicação.

```
podman run -d --name health_test \
--health-cmd='curl -f localhost || exit 1' \
--health-interval=10s \
docker.io/library/nginx

podman inspect --format '{{.State.Health.Log}}' health_test
podman inspect -f '{{json .State.Health}}' health_test
```

■ Estados

starting: aguardando primeiro check. healthy: último check passou. unhealthy: checks consecutivos falharam. curl -f retorna exit não-zero para 4xx/5xx.

S3Q9

Generate Kubernetes YAML

`podman generate kube webapp_pod`

■ Objetivo

Exportar definição do pod para formato Kubernetes portátil.

```
podman generate kube webapp_pod > webapp.yml
cat webapp.yml # começa com: apiVersion: v1 / kind: Pod
```

■ Uso prático

Desenvolver localmente com Podman, exportar para YAML, aplicar em cluster Kubernetes real. O YAML inclui imagens, portas, volumes e configurações dos containers.

S3Q10

Import Kubernetes YAML

`play kube + kube down`

■ Objetivo

Criar pod a partir de YAML Kubernetes e saber como destruí-lo corretamente.

```
podman pod rm -f webapp_pod
podman play kube webapp.yml
podman pod ps

# Teardown correto:
podman kube down webapp.yml
```

■ kube down vs pod rm

kube down remove exatamente o que o YAML criou (pod + containers + volumes). pod rm remove apenas o pod e seus containers.

S3Q11 Remote Image Inspection with Skopeo

Architecture da alpine

■ Objetivo

Inspeccionar imagem remota sem fazer pull para o storage local.

```
skopeo inspect docker://docker.io/library/alpine:latest | grep 'Architecture'
```

■■ Prefixo **docker://** é **OBRIGATÓRIO**. Skopeo é multi-transport: **docker://** = registry remoto, **docker-archive:** = arquivo tar local, **containers-storage:** = storage local do Podman. Sem o prefixo, o comando falha.

S3Q12 Copy Images Between Transports

docker-archive:/tmp/alpine.tar

■ Objetivo

Copiar imagem de registry para arquivo tar local usando Skopeo.

```
skopeo copy docker://docker.io/library/alpine:latest \
  docker-archive:/tmp/alpine.tar
ls -lh /tmp/alpine.tar
```

■ Vantagem sobre podman save

Skopeo copia sem fazer pull para o storage local. Útil para transferências diretas registry → arquivo sem ocupar espaço intermediário no Podman.

S3Q13

Multi-Stage Build Optimization

tiny-app:1.0, golang + ubi-minimal

■ Objetivo

Imagem final pequena que descarta ferramentas de build.

```
FROM docker.io/library/golang:latest AS builder
RUN mkdir /build && touch /build/app

FROM registry.access.redhat.com/ubi9/ubi-minimal
COPY --from=builder /build/app /app

podman build -t tiny-app:1.0 .
```

■ Por que multi-stage

A imagem golang tem centenas de MB de compiladores. A imagem final só precisa do binário. COPY --from=builder pega apenas o necessário. Ferramentas de build não entram na imagem final.

S3Q14

ENTRYPOINT vs CMD

echoer:1.0, exec form obrigatório

■ Objetivo

Entender ENTRYPOINT fixo + CMD como argumento padrão sobrescritível.

```
FROM registry.access.redhat.com/ubi9/ubi
ENTRYPOINT ["echo"]          # fixo – sempre executa
CMD ["Default Message"]     # padrão – sobrescrito em runtime

podman run --rm echoer:1.0           # Output: Default Message
podman run --rm echoer:1.0 'Overridden' # Output: Overridden
```

■ ■ Use SEMPRE exec form ["echo"], NUNCA shell form (echo). Shell form quebra a interação ENTRYPOINT+CMD porque envolve em /bin/sh -c.

S3Q15

Capture Container Death Events

`--filter event=died → death.log`

■ Objetivo

Log em tempo real de containers que terminaram com erro.

```
podman events --stream --filter event=died > death.log &
podman run --rm --name victim \
    registry.access.redhat.com/ubi9/ubi echo 'bye'
cat death.log
fg # Ctrl+C
```

■ died vs exited

died = saiu com `exit ≠ 0` (crash, kill, OOM). exited = saída limpa (`exit = 0`). Para forensics, filtre died para encontrar terminações inesperadas.

S4

Networking Avançado & Segurança

Simulado 4 · 15 Questões

S4Q1

Custom Gateway

app_lan, 10.100.1.0/24, gateway .254

■ Objetivo

Rede com gateway não-padrão para roteamento customizado corporativo.

```
podman network create --subnet 10.100.1.0/24 --gateway 10.100.1.254 app_lan
podman network inspect --format '{{ (index .Subnets 0).Gateway }}' app_lan
```

■ Explicação

Por padrão, o gateway seria .1. Definir .254 é convenção comum em ambientes corporativos. O gateway é o IP que o container usa para tráfego externo à rede.

S4Q2

Multi-Homed Container

dual_nic em app_lan + mgmt_lan via hot-plug

■ Objetivo

Container conectado a duas redes simultaneamente sem reiniciar.

```
podman network create --subnet 192.168.10.0/24 mgmt_lan
podman run -d --name dual_nic --net app_lan \
    registry.access.redhat.com/ubi9/ubi-minimal sleep 3600

podman network connect mgmt_lan dual_nic # hot-plug
podman exec dual_nic cat /etc/hosts      # 2 IPs listados
```

■ Como funciona

podman network connect adiciona segunda interface de rede ao container EM EXECUÇÃO — como plugar um segundo cabo de rede. O container fica visível em ambas as redes simultaneamente.

S4Q3

Troubleshoot Protocol Mismatch UDP

DNS porta 53, rootful

■ Objetivo

DNS precisa UDP na porta 53 — padrão do Podman é TCP.

```
# /udp explícito + sudo para porta privilegiada
sudo podman run -d --name dns -p 53:53/udp \
  registry.access.redhat.com/ubi9/ubi sleep 3600

# ATENÇÃO: use 'sudo podman ps' para ver containers rootful!
```

■ Dois problemas

1. -p 53:53 sem protocolo = TCP por padrão. DNS usa UDP. Solução: /udp explícito. 2. Porta 53 é privilegiada (< 1024) = precisa sudo para rootful.

S4Q4

Deploy from Kubernetes YAML Manifest

db_pod.yaml, mariadb, 13306:3306

■ Objetivo

Criar pod completo a partir de YAML Kubernetes com portmap customizado.

```
# db_pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: db-pod
spec:
  containers:
  - name: db
    image: docker.io/library/mariadb
    env:
    - name: MARIADB_ROOT_PASSWORD
      value: redhat
    ports:
    - containerPort: 3306
      hostPort: 13306

podman play kube db_pod.yaml
podman port db-pod-db # nome: [Pod]-[Container]
```

■ Nome gerado

[NomePod]-[NomeContainer] = db-pod-db. Importante para podman exec, podman logs, etc.

S4Q5**Auto-Start Kubernetes Pod with Quadlet (.kube)****.kube file, PATH ABSOLUTO obrigatório****■ Objetivo**

Pod Kubernetes iniciando no boot via Quadlet .kube file.

```
mkdir -p ~/.config/containers/systemd

# ~/.config/containers/systemd/db.kube
[Kube]
Yaml=/home/student/db_pod.yaml # PATH ABSOLUTO

systemctl --user daemon-reload
systemctl --user enable --now db.service
loginctl enable-linger student
```

■ ■ **Yaml= deve ser PATH ABSOLUTO (/home/student/...). Systemd não entende ~/ ou ./.** Nome do serviço = [arquivo].kube → db.service.

S4Q6**Inject Secrets at Custom Path****api_key → /etc/app/config/key.txt****■ Objetivo**

Secret montado no path exato que a aplicação espera, não o padrão.

```
printf '12345-ABCDE' | podman secret create api_key -
podman run -d --name secure_app \
  --secret api_key,target=/etc/app/config/key.txt \
  registry.access.redhat.com/ubi9/ubi-minimal sleep 3600

podman exec secure_app cat /etc/app/config/key.txt
```

■ **target= redireciona o secret para o path que a aplicação espera. Sem target, o padrão é /run/secrets/. Útil quando não pode alterar o código da aplicação.**

S4Q7

Harden Image - Remove ALL SUID Binaries

`find -perm /u=s, chmod u-s`

■ Objetivo

Remover todos os binários setuid para reduzir superfície de ataque.

```
FROM registry.access.redhat.com/ubi9/ubi
RUN find / -perm /u=s -exec chmod u-s {} + 2>/dev/null || true
RUN useradd appuser
USER appuser

podman run --rm hardened:1.0 find / -perm /u=s 2>/dev/null
# Deve retornar vazio
```

■ ■ **chmod u-s deve rodar ANTES do USER — root tem permissão para modificar /usr/bin. Depois do USER, não há privilégios para isso.**

S4Q8

Share Process Namespace

`debug_pod --share pid`

■ Objetivo

Container debugger enxerga processos de outro container no mesmo pod.

```
podman pod create --name debug_pod --share pid
podman run -d --name database --pod debug_pod \
  -e MARIADB_ROOT_PASSWORD=redhat docker.io/library/mariadb
podman run -d --name debugger --pod debug_pod \
  registry.access.redhat.com/ubi9/ubi sleep 3600

podman exec debugger ps aux | grep mariadb
```

■ --share pid

Faz todos os containers do pod compartilharem o namespace PID. O debugger pode ver e interagir com processos do database. Essencial para debugging em produção sem modificar a aplicação principal.

S4Q9

Limit Container Disk I/O

`--device-read-bps, rootful`

■ Objetivo

Limitar taxa de leitura de disco para evitar que container monopolize I/O.

```
sudo podman run -d --name slow_io \  
  --device-read-bps /dev/vda:1mb \  
  registry.access.redhat.com/ubi9/ubi sleep 3600  
  
sudo podman exec slow_io \  
  dd if=/dev/vda of=/dev/null bs=1M count=10 iflag=direct
```

■ Por que rootful

Limites de I/O usam cgroups v2 com controles de dispositivo que requerem privilégios elevados. `--device-read-bps` limita bytes por segundo de leitura do dispositivo especificado.

S4Q10

Build from Tarball Context

`stdin pipe, tar-build:1.0`

■ Objetivo

Build passando contexto comprimido via stdin, sem criar diretórios temporários.

```
cat source.tar.gz | podman build -t tar-build:1.0 -  
podman images | grep tar-build
```

■ Como funciona

O - no final indica que o contexto vem do stdin. O tar deve conter o Containerfile na raiz. Útil em pipelines CI/CD para evitar arquivos temporários no disco.

S4Q11 Remote Image Inspection Skopeo

Digest da ubi9

■ Objetivo

Obter o digest (hash único) de uma imagem remota sem fazer pull.

```
skopeo inspect docker://registry.access.redhat.com/ubi9/ubi:latest | grep Digest
```

■ Por que Digest

SHA256 do manifest identifica uma versão específica de forma imutável. Mesmo que a tag latest seja atualizada, o digest muda. Garante reprodutibilidade de builds.

S4Q12 Optimize Image Layers

chain && + dnf clean all em um RUN

■ Objetivo

Reduzir tamanho da imagem encadeando comandos em único RUN.

```
FROM registry.access.redhat.com/ubi9/ubi
RUN dnf install -y git wget tar && dnf clean all

podman build -t lean:1.0 .
podman history lean:1.0
```

■ Por que uma layer é melhor

Cada RUN cria nova layer. Se dnf clean all está em RUN separado, a layer do install permanece no histórico. Encadeando com &&, install e cleanup ficam na mesma layer — imagem final menor.

S4Q13 ENTRYPOINT vs CMD (Pinger)

ping com target override

■ Objetivo

ENTRYPOINT como ferramenta fixa, CMD como argumento padrão sobrescritível.

```
FROM registry.access.redhat.com/ubi9/ubi
RUN dnf install -y iputils && dnf clean all
ENTRYPOINT ["ping", "-c", "2"]
CMD ["localhost"]

podman run --rm pinger:1.0          # ping -c 2 localhost
podman run --rm pinger:1.0 google.com # ping -c 2 google.com
```

■ **ENTRYPOINT define o que fazer (ping), CMD define o alvo padrão (localhost) que pode ser trocado facilmente em runtime.**

S4Q14 Container as Build Environment

GCC toolchain, --rm -u 0 -v -w

■ Objetivo

Usar container com toolchain específico para compilar código do host.

```
echo 'int main(){return 0;}' > hello.c
podman run --rm -u 0 -v $(pwd):/src:Z -w /src \
  registry.redhat.io/rhel9/gcc-toolset-12-toolchain \
  gcc hello.c -o hello
```

■ Flags importantes

-u 0 = root dentro do container (para escrever o binário compilado). -v \$(pwd):/src:Z = monta diretório atual. -w /src = define workdir. --rm = remove após compilar. Binário aparece no diretório atual.

S4Q15 **Fix Container Health Check**

porta errada, diagnóstico + correção

■ Objetivo

Diagnosticar health check quebrado e corrigir a configuração.

```
podman inspect --format '{{.State.Health.Log}}' web
podman rm -f web
podman run -d --name web \
  --health-cmd 'curl -f localhost || exit 1' \
  --health-interval=10s \
  docker.io/library/nginx
podman ps # aguardar (healthy)
```

■ Workflow

1. inspect Health.Log para ver o erro. 2. Identificar a causa. 3. rm -f para remover o quebrado. 4. Recriar com health check corrigido.

S5

Troubleshooting Avançado

Simulado 5 · 15 Questões

S5Q1

Missing Host Directory

Bind Mount, /opt/app/logs não existe

■ Objetivo

Diagnosticar e corrigir bind mount para diretório inexistente no host.

```
podman logs logs # stat /opt/app/logs: no such file or directory

sudo mkdir -p /opt/app/logs
sudo chown student:student /opt/app/logs
podman run -d --name logs \
  -v /opt/app/logs:/var/log/app:Z \
  registry.access.redhat.com/ubi9/ubi sleep 3600
```

■ Docker vs Podman

Docker cria o diretório automaticamente no bind mount. Podman NÃO — se não existir, o container falha com erro. A flag -p do mkdir cria toda a hierarquia de diretórios de uma vez.

S5Q2

Fix Broken Containerfile

COPY antes de USER — order matters!

■ Objetivo

Corrigir ordem de instruções — operações root devem vir antes do USER.

```
# Containerfile CORRIGIDO:
FROM registry.access.redhat.com/ubi9/ubi
COPY index.html /var/www/html/ # root tem permissão
USER 1001                       # switch DEPOIS

podman build -t fixed-web:1.0 .
```

■ Regra

Containerfile é executado top-to-bottom. Se USER vem antes do COPY, o usuário 1001 não tem permissão para escrever em /var/www/html/. Qualquer operação que precisa de root deve vir ANTES do USER.

S5Q3

Troubleshoot Pod Communication

WordPress usa localhost, não hostname

■ Objetivo

Corrigir comunicação entre WordPress e banco de dados dentro de um pod.

```
podman pod create --name stack_pod -p 8080:80
podman run -d --pod stack_pod --name db \
  -e MYSQL_ROOT_PASSWORD=redhat docker.io/library/mariadb
podman run -d --pod stack_pod --name wp \
  -e WORDPRESS_DB_HOST=localhost \
  -e WORDPRESS_DB_PASSWORD=redhat docker.io/library/wordpress
```

■ Dentro de um Pod, containers compartilham namespace de rede e comunicam via LOCALHOST — não por nome de container. O hostname db_host não resolve dentro de um pod. DNS por nome só funciona em REDES CUSTOM.

S5Q4

Troubleshoot SELinux Permission Denied

:Z vs :z — não use setenforce!

■ Objetivo

Diagnosticar negação SELinux e corrigir com relabeling de bind mount.

```
sudo ausearch -m AVC # identificar o deny

podman run -d --name secure_write \
  -v ~/data:/data:Z \
  registry.access.redhat.com/ubi9/ubi \
  sh -c 'echo test > /data/test'
cat ~/data/test
```

■ NUNCA use setenforce 0 — desabilita SELinux globalmente, perde pontos no exame. Use :Z (privado para este container) ou :z (compartilhado entre containers).

S5Q5

Harden Image - Remove SUID

passwd + su, chmod u-s

■ Objetivo

Remover setuid de binários específicos para hardening.

```
FROM registry.access.redhat.com/ubi9/ubi
RUN chmod u-s /usr/bin/passwd /usr/bin/su
RUN useradd appuser
USER appuser

# Verificar - -rwxr-xr-x (sem 's')
podman run --rm hardened:1.0 ls -l /usr/bin/passwd /usr/bin/su
```

■ Por que passwd e su

Normalmente têm setuid — precisam de root para modificar /etc/shadow. Em containers onde usuários não trocam senhas, remover esse bit aumenta a segurança.

S5Q6

Fix Name Resolution Failure

rede padrão sem DNS → rede custom

■ Objetivo

Mover containers para rede custom para habilitar resolução DNS por nome.

```
podman rm -f web api
podman network create app_net
podman run -d --name web --net app_net \
    registry.access.redhat.com/ubi9/ubi sleep 3600
podman run -d --name api --net app_net \
    registry.access.redhat.com/ubi9/ubi sleep 3600
podman exec web getent ahosts api
```

■ Diagnóstico

Se getent ahosts retorna vazio ou erro, containers estão na rede padrão. Recriar na rede custom onde o DNS interno do Podman funciona por nome.

S5Q7

Troubleshoot Quadlet Service

arquivo errado + [Install] faltando

■ Objetivo

Corrigir dois problemas comuns com Quadlet: localização e seção [Install].

```
mkdir -p ~/.config/containers/systemd
mv ~/myservice.container ~/.config/containers/systemd/

# Adicionar ao arquivo:
[Install]
WantedBy=default.target

systemctl --user daemon-reload
systemctl --user enable --now myservice.service
systemctl --user status myservice.service
```

■ 1. Arquivo em lugar errado — deve estar em `~/.config/containers/systemd/`. 2. Sem [Install] WantedBy=default.target, o systemctl enable falha silenciosamente.

S5Q8

Fix Build Dependency Order

git: command not found → instalar primeiro

■ Objetivo

Instalar dependência antes de usá-la no Containerfile.

```
FROM registry.access.redhat.com/ubi9/ubi
RUN dnf install -y git && \
    git clone https://github.com/example/repo /app && \
    dnf clean all

podman build -t git-app:1.0 .
```

■ Regra

Containerfiles são executados top-to-bottom. RUN git clone antes de RUN dnf install git resulta em git: command not found. Encadear no mesmo RUN garante também layer menor.

S5Q9**Troubleshoot play kube Failure**

typo image + hostPort < 1024

■ Objetivo

Corrigir YAML Kubernetes com erros de imagem e porta privilegiada.

```
# Antes (quebrado):  
#   image: nginxx           (typo)  
#   hostPort: 80           (porta privilegiada)  
  
# Depois (corrigido):  
#   image: docker.io/library/nginx  
#   hostPort: 8080  
  
podman play kube broken.yaml  
podman pod ps
```

■ Dois problemas distintos

1. nginxx → imagem não existe → pull falha → pod não sobe. 2. hostPort: 80 → usuário rootless não pode usar porta < 1024. Solução: sempre >= 1024 para rootless.

S5Q10**Fix Failing Health Check**

app em :5000, probe em :80

■ Objetivo

Corrigir health check que aponta para porta errada.

```
podman inspect --format '{{.State.Health.Log}}' myapp  
# Erro: curl: (7) Failed to connect to localhost port 80  
  
podman rm -f myapp  
podman run -d --name myapp \  
  --health-cmd 'curl -f localhost:5000 || exit 1' \  
  myimage  
podman ps # aguardar (healthy)
```

■ Diagnóstico

O log do health check mostra Connection refused na porta 80, mas a aplicação escuta em 5000. Corrija --health-cmd para usar a porta correta.

S5Q11 Fix Secret Mounting (Custom Path)`target= /app/secret.conf`**■ Objetivo**

Secret montado no path específico que a aplicação lê.

```
printf 'conf=true' | podman secret create app_conf -
podman run -d --name secure_app \
  --secret app_conf,target=/app/secret.conf \
  registry.access.redhat.com/ubi9/ubi sleep 3600
podman exec secure_app cat /app/secret.conf
```

■ **target= define exatamente onde o secret aparece. Sem ele, o path padrão seria /run/secrets/app_conf. Use quando a aplicação lê de um path fixo que você não pode alterar.**

S5Q12 Troubleshoot Bind Mount Shadowing`Nginx 403, diretório vazio sobrescreve HTML`**■ Objetivo**

Resolver 403 do Nginx causado por bind mount vazio que oculta conteúdo da imagem.

```
mkdir -p ~/host_dir
echo 'Hello' > ~/host_dir/index.html
podman run -d --name web -p 8080:80 \
  -v ~/host_dir:/usr/share/nginx/html:Z \
  docker.io/library/nginx
curl localhost:8080 # Output: Hello
```

■ Diferença bind vs named volume

Bind Mount SOBRESCREVE conteúdo — se diretório do host está vazio, o nginx não tem index.html → 403. Named Volume COPIA arquivos da imagem para o volume na primeira execução.

S5Q13**Troubleshoot Registry Push TLS**

instalar certificado, não --tls-verify=false

■ Objetivo

Configurar certificado corretamente para push seguro ao registry.

```
sudo mkdir -p /etc/containers/certs.d/localhost:5000
sudo cp domain.crt /etc/containers/certs.d/localhost:5000/ca.crt
podman push localhost:5000/myimage
```

■■ **Diretório: /etc/containers/certs.d/ — nome exato do registry. Arquivo: ca.crt. Se pedirem configuração segura no exame, instale o certificado. --tls-verify=false é apenas para lab.**

S5Q14**Monitor Container Events Forensics**

event=died, podman kill

■ Objetivo

Capturar eventos de morte de container para análise forense de incidentes.

```
podman events --stream --filter event=died > events.log &
podman run -d --name test_kill \
  registry.access.redhat.com/ubi9/ubi sleep 3600
podman kill test_kill
cat events.log
fg # Ctrl+C
```

■ Interpretação

died = processo morreu com exit ≠ 0 (crash, kill, OOM). exited = saída limpa. stopped = parado por podman stop. Para forensics, filtre died para terminações inesperadas.

S5Q15

Rootless Storage UID Mapping

podman unshare, UID 1000, ~/data

■ Objetivo

Entender e configurar o mapeamento de UID em ambientes rootless.

```
mkdir -p ~/data
podman unshare chown 1000:1000 ~/data
podman run --rm -u 1000 -v ~/data:/data:Z \
    registry.access.redhat.com/ubi9/ubi touch /data/success
ls -ln ~/data
# owner: número alto (ex: 100999) – UID shift esperado
```

■ Por que o UID fica alto no host

No namespace rootless, UID 1000 dentro do container mapeia para UID_inicial + 999 no host (dependendo do /etc/subuid). podman unshare chown faz o chown dentro do namespace correto.

S6

Troubleshooting & Avançado

Simulado 6 · 15 Questões

S6Q1

Pod Port Mismatch

postgres PGPORT=5433, webapp DB_PORT=5433

■ Objetivo

PostgreSQL configurado para porta não-padrão — webapp precisa estar sincronizado.

```
podman inspect database | grep PGPORT # confirmar porta
podman pod rm -f app_pod
podman pod create --name app_pod
podman run -d --pod app_pod --name database \
  -e POSTGRES_PASSWORD=redhat -e PGPORT=5433 \
  docker.io/library/postgres
podman run -d --pod app_pod --name webapp \
  -e DB_PORT=5433 \
  registry.access.redhat.com/ubi9/ubi-minimal sleep 3600
```

■■ PostgreSQL e MariaDB crasham imediatamente sem POSTGRES_PASSWORD ou MARIADB_ROOT_PASSWORD — o container não sobe.

S6Q2

Broken Network Isolation

client na rede padrão, não secure_net

■ Objetivo

Diagnosticar container no isolamento de rede errado e corrigir.

```
podman inspect client | grep -A2 Networks
# Mostra: 'podman' em vez de 'secure_net'

podman rm -f client
podman run -d --name client --net secure_net \
  registry.access.redhat.com/ubi9/ubi-minimal sleep 3600

# Air-gapped: getent ahosts server
# Com rede: microdnf install -y iputils && ping -c 1 server
```

■■ Usa microdnf, NÃO dnf. Tentar dnf resulta em command not found — esperado e não é bug.

S6Q3

SELinux Port Denial`semanage port -a -t http_port_t -p tcp 8000`**■ Objetivo**

Nginx não consegue bind na porta 8000 por política SELinux — corrigir permanentemente.

```
sudo ausearch -m AVC -ts recent # confirmar AVC name_bind
sudo semanage port -a -t http_port_t -p tcp 8000
# Se 'Port already defined': usar -m (modify)
sudo semanage port -m -t http_port_t -p tcp 8000

podman run -d --name web -p 8000:80 docker.io/library/nginx
curl localhost:8000
```

■■ **NUNCA** `setenforce 0`. `semanage port` adiciona a porta 8000 ao tipo `http_port_t`, permitindo bind por processos web. `-a` para adicionar, `-m` para modificar se já existir.

S6Q4

Optimize Inefficient Containerfile`layer chaining, optimized-app:1.0`**■ Objetivo**

Refatorar múltiplos RUN em único encadeado para imagem menor.

```
FROM registry.access.redhat.com/ubi9/ubi
RUN dnf install -y git && \
    git clone https://example.com/repo /app && \
    dnf clean all

podman build -t optimized-app:1.0 .
podman history optimized-app:1.0
```

■ Por que importa

Layer separada para `dnf clean all` não remove o cache da layer anterior. Encadeando, o espaço do cache não persiste na imagem final. `podman history` mostra o tamanho de cada layer.

S6Q5

Fix Broken Entrypoint Script

--entrypoint /bin/sh para debug

■ Objetivo

Container crasha por erro de sintaxe no shell script de entrypoint.

```
# Bypass do entrypoint quebrado
podman run --rm -it --entrypoint /bin/sh broken-image
# Dentro: cat /usr/local/bin/entry.sh
# Identificar: falta 'fi' fechando um 'if'
# exit

echo 'fi' >> entry.sh
podman build -t fixed-image:1.0 .
podman run --rm fixed-image:1.0 start
```

■ **--entrypoint /bin/sh** *bypassa completamente o script quebrado e abre shell interativo para inspecionar o container. Diferente de CMD override — deve usar a flag --entrypoint.*

S6Q6

Manual Build with Buildah

from/run/copy/config/commit

■ Objetivo

Construir imagem sem Containerfile usando comandos Buildah imperativos.

```
ctr=$(buildah from registry.access.redhat.com/ubi9/ubi)
buildah run $ctr -- dnf install -y gcc
buildah copy $ctr main.c /src/main.c
buildah run $ctr -- gcc /src/main.c -o /usr/bin/app
buildah config --cmd '/usr/bin/app' $ctr
buildah commit $ctr manual-image
podman run --rm manual-image
```

■ ■ **Em buildah run, o -- separa os flags do buildah do comando interno. Sem ele, buildah interpreta flags do comando como seus próprios parâmetros.**

S6Q7

FROM scratch Build

gcc -static, binary estático, sem OS

■ Objetivo

Imagem absolutamente mínima — apenas binário estático, zero OS.

```
gcc -static main.c -o my_static_binary
```

```
# Containerfile
FROM scratch
COPY my_static_binary /app
CMD ["/app"]
```

```
podman build -t scratch-app:1.0 .
podman run --rm scratch-app:1.0
```

■ ■ **FROM scratch é imagem vazia — sem shell, sem libc, sem nada. Binary DEVE ser compilado com -static porque não há glibc no container. Sem podman exec -it (sem shell disponível).**

S6Q8

Rootless Port Binding (porta 80)

sysctl permanente + Quadlet

■ Objetivo

Configurar sistema permanentemente para usuário rootless usar porta 80.

```
echo 'net.ipv4.ip_unprivileged_port_start=80' | \
  sudo tee /etc/sysctl.d/99-rootless.conf
sudo sysctl --system
```

```
# web.container:
[Container]
Image=docker.io/library/nginx
PublishPort=80:80
[Install]
WantedBy=default.target
```

```
systemctl --user daemon-reload
systemctl --user enable --now web.service
```

■ ■ **sysctl -w é temporário (perde no reboot). Escrever em /etc/sysctl.d/*.conf é permanente e aplicado com sysctl --system.**

S6Q9**Fix Rootless Host Permission**

--userns=keep-id, host UID = container UID

■ Objetivo

Arquivos criados pelo container pertencem ao usuário do host — sem mapeamento estranho.

```
mkdir -p ~/data
podman run --rm -v ~/data:/data:Z --userns=keep-id \
  registry.access.redhat.com/ubi9/ubi touch /data/test
ls -l ~/data # arquivo pertence ao host user (student)
```

■ keep-id vs unshare chown

--userns=keep-id: UID do container = UID do host. Arquivos criados pertencem ao seu usuário. podman unshare chown: ajusta o host para combinar com o UID do container. São abordagens opostas.

S6Q10**Debug Kubernetes YAML**

remover hostIP + corrigir porta privilegiada

■ Objetivo

Remover configurações não-portáteis e corrigir porta privilegiada em YAML Kubernetes.

```
# Remover:
#   hostIP: 192.168.1.100 (hardcoded – não portátil)

# Corrigir:
#   hostPort: 80 -> hostPort: 8080

podman play kube broken_pod.yaml
podman pod ps
```

■ Boas práticas

hostIP hardcoded torna YAML não portátil entre ambientes. hostPort < 1024 falha para rootless. Ambas são armadilhas comuns no exame.

S6Q11 Remote Inspection Skopeo

Created timestamp

■ Objetivo

Verificar quando uma imagem foi criada sem fazer pull.

```
skopeo inspect docker://registry.access.redhat.com/ubi9/ubi:latest | grep Created
```

■ **Skopeo consulta o registry remotamente e retorna o timestamp de criação do manifest. Útil para verificar se imagem foi atualizada recentemente sem baixá-la. docker:// é OBRIGATÓRIO.**

S6Q12 Troubleshoot Flapping Health Check

timeout muito curto em app lenta

■ Objetivo

Estabilizar health check instável causado por timeout muito curto.

```
podman inspect --format '{{.State.Health.Log}}' web
# timeout / connection refused

podman rm -f web
podman run -d --name web \
  --health-cmd 'curl -f http://localhost || exit 1' \
  --health-timeout 5s \
  --health-retries 3 \
  --health-start-period 60s \
  docker.io/library/nginx
```

■ Parâmetros

--health-timeout: tempo máximo por probe. --health-retries: falhas antes de unhealthy.
--health-start-period: período de graça no início (ignora falhas durante boot da app lenta).

S6Q13 Complex Data Pipeline

Pod + 3 containers + volume compartilhado

■ Objetivo

Pipeline de dados com 3 containers sequenciais compartilhando volume.

```
podman volume create shared_data
podman pod create --name pipeline

# Generator – escreve dados brutos
podman run -d --pod pipeline --name generator \
  -v shared_data:/data:Z registry.access.redhat.com/ubi9/ubi \
  sh -c 'mkdir -p /data/raw && echo 42 > /data/raw/numbers.txt && sleep 3600'

# Processor – processa após 2s
podman run -d --pod pipeline --name processor \
  -v shared_data:/data:Z registry.access.redhat.com/ubi9/ubi \
  sh -c 'sleep 2 && cp /data/raw/numbers.txt /data/processed/even.txt && sleep 3600'

# Archiver – arquiva após 4s
podman run -d --pod pipeline --name archiver \
  -v shared_data:/data:Z registry.access.redhat.com/ubi9/ubi \
  sh -c 'sleep 4 && tar -czf /data/archive.tar.gz -C /data/processed . && sleep 3600'
```

■ *sleep 2s e 4s simulam dependência entre etapas sem orquestração complexa. Em produção seria feito com health checks ou mensageria.*

S6Q14 Audit Container Exec

tripwire, filter event=exec

■ Objetivo

Detectar e registrar qualquer podman exec em container sensível.

```
podman run -d --name secure_app \
  registry.access.redhat.com/ubi9/ubi sleep 3600

podman events --stream --filter event=exec > events.log &
podman exec -it secure_app ls / # simula acesso não autorizado
cat events.log
fg # Ctrl+C
```

■ Caso de uso de segurança

Em ambientes regulados, qualquer exec em container de produção deve ser auditado. --filter event=exec captura timestamp, container ID e o comando executado.

S6Q15**Capstone: Gitea + PostgreSQL**

Pod + Volumes + Secrets + localhost

■ Objetivo

Stack completo reunindo todos os conceitos do exame em uma aplicação real.

```
podman volume create pg_data
podman volume create gitea_data
printf 'redhat123' | podman secret create db_pass -

podman pod create --name gitea_pod -p 3000:3000 -p 2222:22

podman run -d --pod gitea_pod --name gitea_db \
  --secret db_pass \
  -v pg_data:/var/lib/postgresql/data:Z \
  -e POSTGRES_DB=gitea \
  -e POSTGRES_PASSWORD_FILE=/run/secrets/db_pass \
  docker.io/library/postgres

podman run -d --pod gitea_pod --name gitea_web \
  --secret db_pass \
  -v gitea_data:/data:Z \
  -e DB_TYPE=postgres \
  -e DB_HOST=localhost:5432 \
  -e DB_PASSWD_FILE=/run/secrets/db_pass \
  docker.io/gitea/gitea
```

■ Conceitos reunidos

Pod com múltiplas portas, named volumes com :Z, secrets com padrão _FILE (sem expor senha em variável), comunicação via localhost dentro do pod, dois containers com dependência entre si.

■ Checklist Final — 35 Tópicos Críticos

01. ■ Criar container como serviço Quadlet (.container) com [Install] WantedBy=default.target
02. ■ Usar podman generate systemd --new para container e pod
03. ■ Criar Quadlet .kube file com path ABSOLUTO ao YAML Kubernetes
04. ■ Usar loginctl enable-linger para persistência sem login do usuário
05. ■ Criar bind mount com :Z (privado) ou :z (compartilhado) para SELinux
06. ■ Usar podman unshare chown para alinhar UID em bind mounts rootless
07. ■ Usar --userns=keep-id para manter UID do host dentro do container
08. ■ Criar named volume e montar sem :Z (Podman labela automaticamente)
09. ■ Criar imagem com ARG (build-time) + ENV (runtime) + LABEL (metadados)
10. ■ Usar ADD (não COPY) para extrair .tar.gz automaticamente no build
11. ■ Construir multi-stage build com COPY --from=builder
12. ■ Criar e montar secret com target= para path customizado
13. ■ Usar printf (não echo) para criar secrets sem \n invisível
14. ■ Criar rede custom com subnet, gateway customizado e IP estático
15. ■ Usar podman network connect para hot-plug em segunda rede (multi-homed)
16. ■ Configurar DNS entre containers (rede custom — não funciona na padrão)
17. ■ Criar Pod com port mapping — containers filhos NÃO usam -p
18. ■ Usar --share pid para compartilhar namespace de processos entre containers
19. ■ Criar Compose com serviços, volumes nomeados e redes custom
20. ■ Usar podman generate kube, podman play kube e podman kube down
21. ■ Instalar certificado de registry em /etc/containers/certs.d//ca.crt
22. ■ Usar skopeo inspect docker:// e skopeo copy docker-archive: (prefixo obrigatório!)
23. ■ Usar semanage port -a/-m -t http_port_t -p tcp para SELinux de rede
24. ■ Configurar sysctl PERMANENTE em /etc/sysctl.d/*.conf (não -w que é temporário)
25. ■ Usar --entrypoint /bin/sh para debug de container com entrypoint quebrado
26. ■ Construir imagem com buildah from/run/copy/config/commit (sem Containerfile)
27. ■ Criar imagem FROM scratch com binary compilado estaticamente (gcc -static)
28. ■ Chain comandos com && e dnf clean all em RUN único para otimizar layers
29. ■ Usar podman events --stream --filter event=died/exec para forensics
30. ■ Inspecionar health check com podman inspect --format {{.State.Health.Log}}
31. ■ Configurar --health-timeout e --health-retries para health check flapping
32. ■ Usar ausearch -m AVC para diagnosticar SELinux (arquivo e rede)
33. ■ Corrigir ordem no Containerfile: COPY/RUN antes de USER para operações root
34. ■ Resolver Port Conflict mudando hostPort > 1024 em contexto rootless
35. ■ Capstone: Pod + Secrets + Named Volumes + comunicação localhost entre containers